Architecture overview

Frontend (React/TypeScript)

Where: Railway

- Domain: www.ppaltsokas.com
- Auto-deploys on GitHub push
- Environment variable: VITE_API_URL points to the backend
- Built with Vite and served as static files

Backend (FastAPI/Python)

Where: Google Cloud Run

- URL: https://virtual-persona-backend-vw5orx4ptq-oc.a.run.app
- Containerized with Docker
- Manually deployed via deploy-backend.ps1
- Runs the chatbot and serves the CV PDF

Custom domain

Where: Namecheap

- ppaltsokas.com (root domain)
- DNS:
- www → CNAME → Railway frontend
- Root domain → URL Redirect → www.ppaltsokas.com

How requests flow

User visits www.ppaltsokas.com

↓

Railway serves React app (static HTML/JS/CSS)

↓

User clicks "Download Resume" or chats

↓

Frontend makes API call to VITE_API_URL

↓

Request goes to Google Cloud Run backend

↓

Backend processes request:

- Chat: Search KB → Gemini API → Stream response
- CV: Serves PDF from /me/cv endpoint

↓

Response sent back to frontend

↓

Frontend displays results

File locations

Code repository

- GitHub: your repo
- Local: F:\Virtual Persona CV

Knowledge base

- Location: kb/ (in repo and Docker image)
- Contents: PDFs, markdown files, project docs
- Served by backend from within the container

CV file

- Location: me/CV PALTSOKAS PANAGIOTIS.pdf (in repo)
- Served at: /me/cv endpoint (backend)

Environment configuration

Railway (Frontend)

- VITE_API_URL: Backend URL
- Auto-detected from git push

Cloud Run (Backend)

- GEMINI_API_KEY: API key (set during deployment)

- APP_ENV: prod
- CORS: allows www.ppaltsokas.com and ppaltsokas.com

Deployment process

Frontend (automatic)

1. Push to GitHub
2. Railway detects changes
3. Builds React app
4. Deploys automatically

Backend (manual)

1. Run .\deploy-backend.ps1
2. Script builds Docker image
3. Pushes to Google Artifact Registry
4. Deploys to Cloud Run
5. New revision goes live

Key components

1. Frontend (App.tsx, components/) — React UI
2. Backend (main.py.backend) — FastAPI server
3. Knowledge base (kb/) — 150+ documents
4. Chat service (services/geminiService.ts) — Frontend API client
5. Resume data (constants.ts) — Static resume content